Group 47 Progress Report:
Single Class Movie Genre Classifier

Nour Fawaz, Daniel Maurer, Lily Porter, Sriya Dhanvi Mokhasunavisu
{fawazn,maured2,portel7,mokhasus}@mcmaster.ca

1 Introduction

Here, write a brief introduction to the problem you are solving. This can be adapted from your problem description and motivation from the original proposal. This should be around 0.25-0.5 pages. A frequent problem that arises within services that serve large amounts of content to the public is the generation of searchable tags for this content. Some services rely on the creators of the content to create these tags themselves, or may choose to auto-generate these tags, which can be error-prone. It is important that these tags are fairly accurate, as it is often through these tags that users of the service find suitable content to view, either through recommendations or manual searches. A specific example of this problem is the classification of movies into their genre(s), a problem likely faced by streaming platforms. Should these providers choose to auto-generate these tags, the most efficient method involves analysis of the provided movie description. For the process to be automatic, the tag generation algorithm must include some level of natural language processing, which is where the challenge of the task originates. Movie descriptions are often short, vague, and potentially misleading without greater context, despite often being obvious to human readers as to what genre the movie might be. As for the decision of what genre the movie itself belongs to, people generally can agree, although difficulty can result if a movie belongs to multiple genres at once. We aim to create a natural language processing algorithm that would take as input the name and description of a movie, and output its genre to aid in this issue.

2 Related Work

Here, talk about the related work you encountered for your approach. Cite at least 5 references. Refer to item 1. No one has done exactly your task? Write about the most similar thing you can find. This should be around 0.25-0.5 pages.

3 Dataset

You should write about your dataset here, following the guidelines regarding item 2. This section may be 0.5-1 pages. Depending on your specific dataset, you may want to include subsections for the preprocessing, annotation, etc.

4 Features

Describe any features you used for your model, or how your data was input to your model. Are you doing feature engineering or feature selection? Are you learning embeddings? Is it all part of one neural network? Refer to item 3. This may range from 0.25 pages to 0.5 pages.

4.1 Logistic Regression Model

4.1.1 Base Logistic Regression Model

For the Basic Logistic Regression model, the movie plots were used as features and the genres as labels for training. The features were preprocessed and lemmatized to exclude all characters and words that aren't nouns, adjectives, verbs, or adverbs. Next, they were tokenized such that each word is a separate token. Afterwards, they were fed into a skip-gram Word2Vec model with vector size 200, window size of 5, and 30 epochs. Finally, the document embeddings are determined using the word embeddings produced by the model by averaging them into a single 200-dimensional vector. The plots were used as the features as they are the premise of the movie, conveying the plot, setting, and potential characters. These aspects are what generally make up a movie and considering most of these concepts are communicated as nouns, adjectives, adverbs and verbs, they were the only type of words fed into the model.

4.1.2 Variation 1: Logistic Regression with TF-IDF

The first variation of the logistic regression model is the TF-IDF and Additional Postags Model (labelled Variation 1 in the logistic-regression-model.ipynb file). It first consists of modifying the lemmatization process such that it kept proper nouns in addition to the other types of words (nouns, verbs, adverbs, and adjectives). Furthermore, the word representation used is a static TF-IDF bag-of-words model. As can be seen in figure X, this results in both the accuracy and f1-score being reduced by approximately 0.04 and 0.07 respectively. This is likely due to the fact that whereas TF-IDF just counts word frequencies, Word2Vec captures contextual relationships and can understand the similarity between certain words. This is important for movie genre classification considering the same genre will likely have similar contexts thereby allowing the model to capture those relationships.

4.1.3 Variation 2: Logistic Regression with DistilBERT

The second variation of the logistic regression model is the Logistic Regression DistilBERT Model (labelled Variation 2 in the logistic-regression-model.ipynb file). It uses the HuggingFace pre-trained DistilBERT model as input. This model has much richer word representations compared to Word2Vec and TF-IDF. This is clearly reflected in figure X where the f1-score and accuracy increase by approximately 0.08 and 0.06 compared to the Base Model respectively. There are several reasons as to why these metrics experienced a notable increase. First, whereas the Base Word2Vec model was trained only on movie plot dataset, DistilBERT was trained on several resources such as Wikipedia and BookCorpus. Hence DistilBERT already has a deeper language understanding compared to the Base Model. Second, whereas Word2Vec has a single representation for a token, DistilBERT has several static word representations for the same token depending on its context. This is incredibly useful for classifying movie genre plots as several words can be used differently given its context.

5 Implementation

Describe your model and implementation here. Refer to item 4. This may take around a page.

5.1 Logistic Regression Model

The first model created was a Logistic Regression model with skip-gram word embeddings. This is the trained baseline. It has several variations each with different word representations as input with some variations outperforming others. For all models, Sci-kit's built in logistic regression loss function which is cross-entropy loss was used.

5.1.1 Base Logistic Model with Skip-gram

The base model is as mentioned, a Logistic regression model with skip-gram word embeddings. To create the model, first the movie descriptions were lemmatized to remove all words excluding nouns, adjectives, verbs and adverbs. They were then tokenized by whitespace. Next, a Word2Vec model was created using Gensim with input parameters; skip-gram, vector size of 200, window size 5, and 30 epochs. Next, the document embeddings were determined using the word embeddings produced by the model by averaging them into a single 200-dimensional vector. Afterwards, the data was split such that 80% was for training and 20% for testing. Next the Logistic Regression model was instantiated with max iterations of 1000, and fit using the training data. Finally, the data was evaluated on the test set obtaining a weighted f1 score of 0.445 and an accuracy of 0.475. To optimize the model, grid search and stratified 5-fold cross validation was used. The model was input into a GridSearchCV object along with a parameter grid consisting of C values 0.1, 0.9, 1, 1.1, 1.15, and 1.2, and L1 and L2 as options for regularization. The scoring metric used was weighted f1 score. This resulted in a slight increase, concluding the best model consisted of a C value of 1.1, while using L2 regularization. This model outperforms both simple baselines significantly which is expected considering the model is trained, it has knowledge of certain relationships among words within the training set.

5.1.2 Logistic Regression with TF-IDF

The first variation of the model was a logistic regression model with TF-IDF word representations. To create the model, first the movie plot descriptions were preprocessed and lemmatized to remove all words excluding nouns, proper nouns, adjectives, adverbs, and verbs. It was decided to add proper nouns to the vocabulary considering this may signal to a specific genre. For example, there are several Animation movies and many of

them are about the adventures of Winnie the Pooh. Hence, when the model spots the name Winnie the Pooh, it can uniquely capture the relationship between the movies and the genre. After lemmatization, the words were tokenized by whitespace and split into an 80% training set and 20% test set. Next the resulting words were transformed into a TF-IDF word representation using Sci-kit's TfidfVectorizer class. They were then fed into a logistic regression model that was fit on the training data. Finally, the model was evaluated on the test set scoring an f1 score of 0.387 and an accuracy of 0.449. This model preformed worse than the base logistic regression model, and this is expected considering it uses a simple bag-of-words variation as input. Hence, no actual semantic meaning was learned, only word and document frequencies. However, this variation does still perform better than the random and majority baselines. This is considering the model uses word frequency as a metric when evaluating movie plots against genres, rather than random guessing or guessing the same genre.

5.1.3 Logistic Regression with DistilBERT

The second variation created was a logistic regression model with DistilBERT word embeddings. To create the model, first the distilbert-base-cased word embeddings were loaded from Hugging-Face. Notice the cased version was chosen considering this maintains capitalization for words. This is important because the absence of capitalization may result in lost information since the model will not be able to distinguish between the same words that may have completely different meanings. For example, "IT" the clown is not the same as "it" the pronoun. Next, DistilBERT's vocab and weights were loaded using Hugging-Face's AutoTokenizer and AutoModel classes. Afterwards, the model was put into evaluation mode, and the data was split into an 80% training set and a 20% testing set. Next a LabelEncoder was created to map the genres to integers for later evaluation. Then a function that determines the BERT word embeddings for the movie plots was created (get_bert_embeddings). This function first converts the text into tensors in batches of 32 then passes them through the BERT model to create the embeddings. Next a logistic regression model was created and fed the BERT embeddings for training. Finally, it was evaluated with the same metrics resulting in an f1 score of 0.512 and an accuracy of 0.531. This has a significant increase in both the f1 score and accuracy compared to both previous variations (including the majority and random baselines). This is expected considering a pretrained model was used to determine contextual word embeddings that was fed into the logistic regression model for prediction. DistilBERT creates a word representation based on the word's surrounding text. In contrast to skip-gram that only generates static embeddings that has a single fixed vector for a word regardless of its surrounding words.

	Model	F1 (Weighted)	Accuracy
0	Majority Baseline	0.113525343489242 41	0.26831501831150183
1	Random Baseline	0.06797697750182068	0.04761904761904761 6

Figure 1: A figure with a caption that runs for more than one line. Example image is usually available through the `mwe` package without even mentioning it in the preamble.

6 Results and Evaluation

What are your results? What did you implement and what did your classmates implement? What are your baselines? Refer to items 5 and 6. This may take around 0.5-1 pages.

6.1 Evaluation

When assessing the models, the data was divided such that 80% of the data was used for training and 20% for testing. This totals to approximately 4364 training instances and 1092 testing instances. As can be seen in figure Y, the dataset mainly consisted of Drama and Comedy, followed by Action, then Adventure, Horror, and Crime. All following genres have approximately less than 200 instances. Considering the imbalance in the dataset, weighted f1-score was used as the primary metric for evaluation. The secondary evaluation metric was accuracy to measure the proportion of correct predictions of each model.

6.2 Baseline Results

The simple baselines created were the majority class baseline and random class baseline. The majority class baseline was created by always predicting the most common genre in the dataset, which is Drama. The random class baseline was created by randomly predicting a genre for each instance in the dataset. The results of the baselines are shown in the figure X below.

6.3 Logistic Regression Model

One of the approaches taken by a participant is finetuning a DeBERTa (deberta-vs-base) model. They finetuned on 90% of the given data and used the remaining 10% for validation across 10 epochs. They achieved an accuracy of 0.62 and a (weighted) f1 score of 0.59 on the validation set. On the other hand, when testing the model on the provided test set, they achieved a (weighted) f1 score of 0.51 and an accuracy of 0.53. These results were expected because, after constructing a couple of models, and trying different variations for the task, when using DistilBERT word embeddings with a logistic regression model and using it to predict the genres, the same results were obtained for the test set.

Another approach taken by another participant is finetuning a RoBERTa (Roberta-base) model. As can be seen below, the model scored a f1-score of 0.53 and an accuracy of 0.55. This surprisingly performed better than both the DeBERTa and logistic regression DistilBERT models. This may be due to the tokens being more specific to certain contexts or length of the movie plots. Furthermore, this may also be due to the splitting of the data, optimization of the model parameters, etc.

7 Error Analysis

What kind of errors are your models making? This doesn't have to be super complicated. Confusion matrices are great. Qualitative observations are also great. This means looking at many cases where your model is wrong (wrong class or high error) and looking for patterns yourself. What about the text might be giving it difficulty? Qualitative analysis is underappreciated and can often give you great insight into issues with your model. Examples are useful here. Refer to item 7. This may be around a half page. /subsectionLogistic Regression Model A summary of the f1-scores and accuracies produced by the different Logistic Regression Model variations can be observed in figure X. As shown, the DistilBERT model outperforms both the Word2Vec and TF-IDF models by obtaining higher evaluation metrics, whereas the TF-IDF performs the worst. Additionally, in figure Z, one may observe how well each model predicted the genre of a movie for each class. It can first be noted that the DistilBERT model (on the far right) has the brightest (most yellow) diagonal indicating that it had the greatest number of correct classifications. Additionally, labels 0, 4, and 7 have the greatest amount of data likely representing the most common genres in the dataset (Action, Comedy, and Drama). Furthermore, considering the diagonal is brightest for those three genres across all models, this clearly represents that the models are best at predicting these genres, as they are represented the greatest in the dataset. On the other hand, labels such as 8 (Fantasy), 9 (Film-Noir), 11 (IMAX), 12 (Musical), 13 (Mystery), and 14 (Romance) have no predictions across all the models, likely indicating these are under-represented genres in the dataset. This further supports that the model is bad at classifying these genres since, as mentioned previously, they are under-represented in the dataset. Additionally, a pattern in the error is that all the models mis-classify rare genres into the more common ones. For example, predicting crime (5) as comedy (4). This incorrect pattern may be fixed by, collecting more data for the under-represented genres, augmenting the data, or using a weighted loss function.

8 Conclusions

Summarize your work briefly. Reflect on your implementation and areas for improvement. What do you suggest people work on next (mentioned also in item 7)? This should likely be less than 0.25 pages.

8.1 Logistic Regression Model

The overall results of the Logistic Regression model could have been improved. For example, for the second variation of the model, I believe with actually fine-tuning DistilBERT rather than using it to obtain static features, the model could have yielded better results. Yet, the improvement from the base model to the second variation is notable, unlike the decrease in performance from the base to the first variation. Overall, to an individual approaching this problem, I would recommend obtaining a balanced dataset and attempting different data augmentation techniques to allow the model to properly learn aspects of the different genres. This way the model can better correlate words to its context and recognize patterns more efficiently thereby yielding more accurate results for its predictions.

9 Template Notes

You can remove this section or comment it out, as it only contains instructions for how to use this template. You may use subsections in your document

Model	F1 Score (Weighted)	Accuracy
Majority Baseline	0.11	0.27
Random Baseline	0.07	0.05
Base Logistic Regression (Word2Vec)	0.45	0.48
Variation 1: Logistic Regression (TF-IDF)	0.39	0.45
Variation 2: Logistic Regression (DistilBERT)	0.51	0.53
Participant 1: DeBERTa (deberta-vs-base)	0.51	0.53
Participant 2: RoBERTa (Roberta-base)	0.53	0.55

Table 1: Model performance comparison on movie genre classification.

	Model	F1 (Weighted)	Accuracy
0	Base Logistic Regression Model	0.4452439731480292	0.47527472527472525
1	Logistic Regression with TF-IDF	0.3871227998833605	0.44871794871794873
2	Logistic Regression with DistilBERT	0.5124619380107539	0.5311355311355311

Figure 2: Comparison of the f1-scores and accuracies of the different Logistic Regression Model variations.

as you find appropriate.

9.1 Tables and figures

See Table 2 for an example of a table and its caption. See Figure 4 for an example of a figure and its caption.

9.2 Citations

Table 2 shows the syntax supported by the style files. We encourage you to use the natbib styles. You can use the command `\citet` (cite in text) to get "author (year)" citations, like this citation to a paper by Gusfield (1997). You can use the command `\citep` (cite in parentheses) to get "(author, year)" citations (Gusfield, 1997). You can use the command `\citealp` (alternative cite without parentheses) to get "author, year" citations, which is useful for using citations within parentheses (e.g. Gusfield, 1997).

9.3 References

Many websites where you can find academic papers also allow you to export a bib file for citation or bib formatted entry. Copy this into the `custom.bib` and you will be able to cite the paper in the LaTeX. You can remove the example entries.

9.4 Equations

An example equation is shown below:

$$A = \pi r^2 \tag{1}$$

Labels for equation numbers, sections, subsections, figures and tables are all defined with the `\label{label}` command and cross references to them are made with the `\ref{label}` command. This an example cross-reference to Equation 1. You can also write equations inline, like this: $A = \pi r^2$.

Team Contributions

Write in this section a few sentences describing the contributions of each team member. What did each member work on? Refer to item 8.

References

Rie Kubota Ando and Tong Zhang. 2005. A framework for learning predictive structures from multiple tasks and unlabeled data. *Journal of Machine Learning Research*, 6:1817–1853.

Galen Andrew and Jianfeng Gao. 2007. Scalable training of L1-regularized log-linear models. In *Proceedings of the 24th International Conference on Machine Learning*, pages 33–40.

Dan Gusfield. 1997. *Algorithms on Strings, Trees and Sequences*. Cambridge University Press, Cambridge, UK.

Mohammad Sadegh Rasooli and Joel R. Tetreault. 2015. Yara parser: A fast and accurate dependency parser. *Computing Research Repository*, arXiv:1503.06733. Version 2.

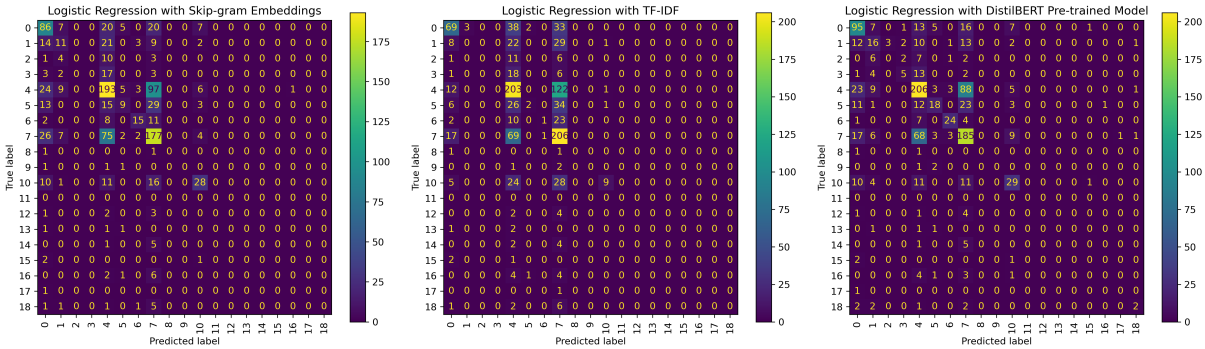

Figure 3: Confusion matrices for the different Logistic Regression Model variations.

Output	**natbib command**	**ACL only command**
(Gusfield, 1997)	`\citep`	
Gusfield, 1997	`\citealp`	
Gusfield (1997)	`\citet`	
(1997)	`\citeyearpar`	
Gusfield's (1997)		`\citeposs`

Table 2: Citation commands supported by the style file.

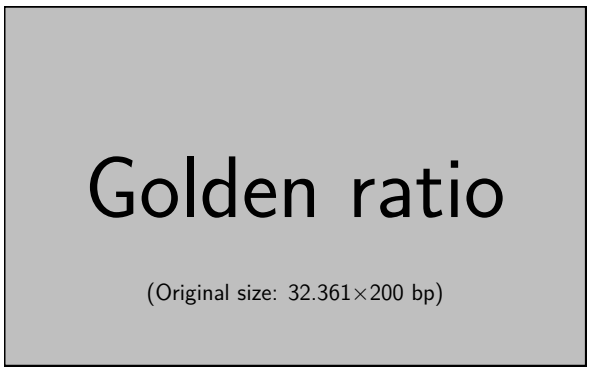

Figure 4: A figure with a caption that runs for more than one line. Example image is usually available through the `mwe` package without even mentioning it in the preamble.

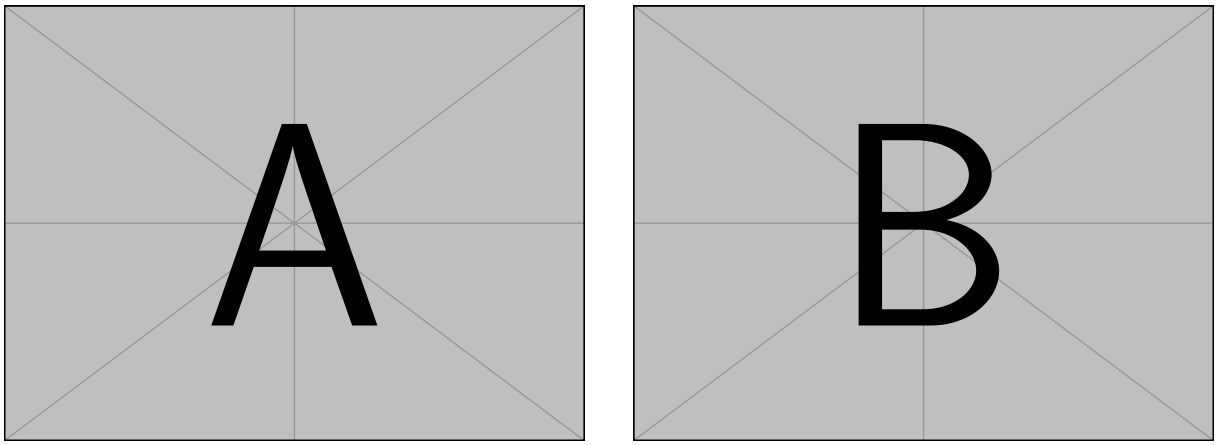

Figure 5: A minimal working example to demonstrate how to place two images side-by-side.